

HEAVEN'S LIGHT IS OUR GUIDE



RAJSHAHI UNIVERSITY OF ENGINEERING & TECHNOLOGY

DEPARTMENT OF MECHATRONICS ENGINEERING, 21Series

Course No: MTE 3104

Course Title: Microcontroller and Interfacing Sessional

Experiment No: 10

Experiment Name: Interfacing LM35 Temperature sensor using Atmega32 with proteus simulation.

Submitted To:

Md. Sakib Hassan

Lecturer,

Department of Mechatronics Engineering,
Rajshahi University of Engineering & Technology

Md Zunaid Hossen

Lecturer,

Department of Mechatronics Engineering,
Rajshahi University of Engineering & Technology

Submitted by:

Adiba Binta Newaz

2108043

Date of Submission:

28th June, 2025

Experiment No: 10

Experiment Name: Interfacing LM35 Temperature sensor using Atmega32 with proteus simulation.

Objectives:

1. To understand the working principle and interfacing of the LM35 temperature sensor with a microcontroller.
2. To learn how to use Atmega32 microcontroller for analog to digital conversion (ADC).
3. To simulate the hardware setup using Proteus software before actual implementation.

Theory:

The LM35 temperature sensor is a precision IC that provides an output voltage linearly proportional to the Celsius temperature. It doesn't require any external calibration and offers a scale factor of 10 mV per degree Celsius. This makes it easy to interface with microcontrollers via ADC.[1]

An LCD display, typically a 16x2 character-based module, is used to show real-time temperature readings. It connects to the microcontroller via its data and control pins and requires initialization in 4-bit or 8-bit mode. This experiment combines all these components into a system where the Atmega32 reads temperature from the LM35 using its ADC, processes the data, and displays it on the LCD. The simulation is performed in Proteus, and the firmware is developed in Atmel Studio.

Required Software:

1. Atmel Studio 7
2. Proteus 8

Code:

```
#define SAMPLE 32
#include <avr/io.h>
#include <util/delay.h>
#include "lcd_lib.h"
static uint16_t ReadADC(uint8_t chn);
int main(void) {

    _delay_ms(100);

    uint8_t vin_i = 0, vin_f = 0, temperature = 0;
    uint16_t adcsres = 0, adc_lm35 = 0;
```

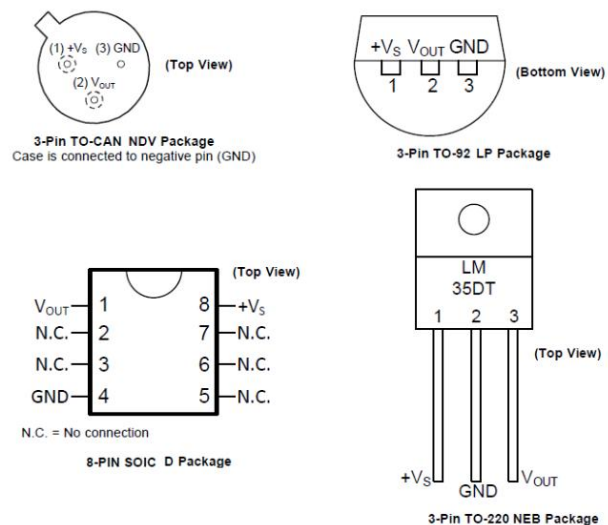


Figure 1: LM35 Temperature Sensor

```
double vin = 0.0;

    ADMUX = 0x40;
    ADCSRA = 0x83;

    LCDinit();
    LCDWriteStringXY(0, 0, "ADC Example");
    _delay_ms(2000);

    while (1) {
        adcsres = 0;
        for (uint8_t s = 0; s < SAMPLE; s++) {
            adcsres += ReadADC(0);
        }
        adcsres = adcsres/SAMPLE;
```

```

vin = (double) ((adcsres * 5.0)/1024);

vin_i = (uint8_t) vin;
vin_f = (uint8_t)((vin - vin_i) * 100);

LCDWriteIntXY(2, 1, vin_i, 1);
LCDWriteString(".");
LCDWriteIntXY(4, 1, vin_f, 2);
LCDWriteStringXY(7, 1, "V");
_delay_ms(100);
adc_lm35 = 0;
for (uint8_t s = 0; s < SAMPLE; s++) {
    adc_lm35 += ReadADC(1);
}
adc_lm35 = adc_lm35/SAMPLE;

temperature = (uint8_t) (adc_lm35/2);

LCDWriteIntXY(9, 1, temperature, 2);
LCDWriteStringXY(12, 1, "*C");
_delay_ms(100);
}
}

static uint16_t ReadADC(uint8_t chn) {

    uint16_t result = 0;
    if (chn > 7)
        return result;
    ADMUX |= (chn & 0x07);

    ADCSRA |= (1<<ADSC);

    while( (ADCSRA & (1<<ADSC)));

    result = ADCL;
    result |= (uint16_t) (ADCH<<8);

    ADMUX &= ~0x1F;

    return result;
}

```

Code Explanations:

1. SAMPLE 32: Number of samples to average for accurate ADC readings.
2. lcd_lib.h: Custom LCD library for initializing and writing data to an LCD.
3. static uint16_t ReadADC(uint8_t chn); Declares a static function to read 10-bit ADC data from the specified channel.
4. Initial 100 ms delay stabilizes LCD; variables store ADC results and processed voltage/temperature; ADMUX = 0x40 sets AVCC ref & channel 0; ADCSRA = 0x83 enables ADC with prescaler 8.
5. Initializes LCD, displays "ADC Example" on row 0, and waits 2 seconds.
6. Reads 32 samples from ADC channel 0, averages them, converts to voltage, splits into integer and decimal parts, and displays as X.XXV on LCD at row 1, column 2.
7. Reads 32 samples from channel 1 (LM35), averages them, converts to temperature using ADC/2, and displays result as XX°C on LCD at row 1, column 9.
8. Reads 10-bit ADC value from selected channel (0–7) by setting ADMUX, starting conversion, waiting for completion, combining ADCL and ADCH, clearing channel bits, and returning the result.

Circuit Diagram:

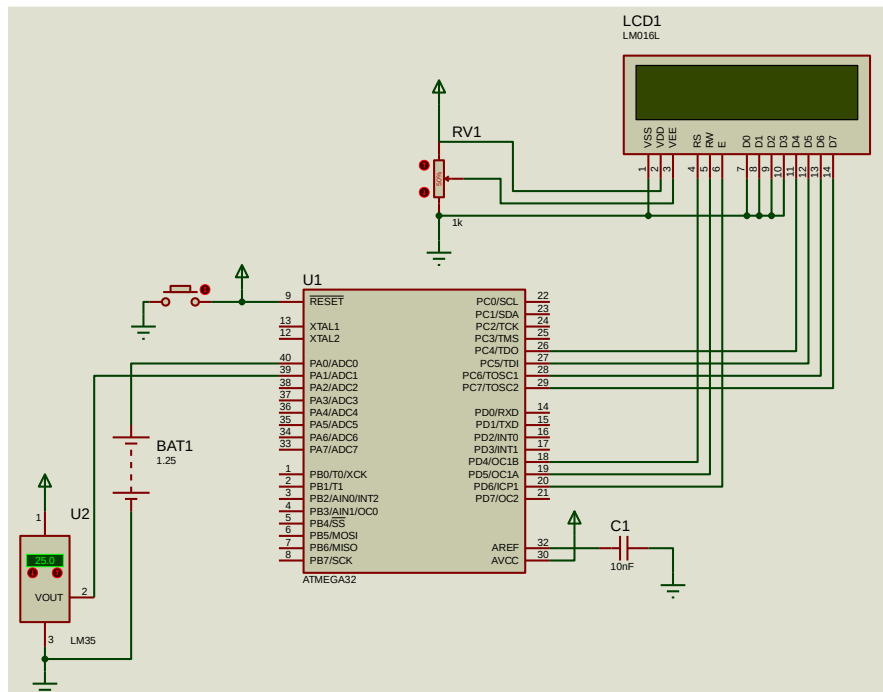


Figure 2: LM35 Temperature sensor circuit diagram

Result:

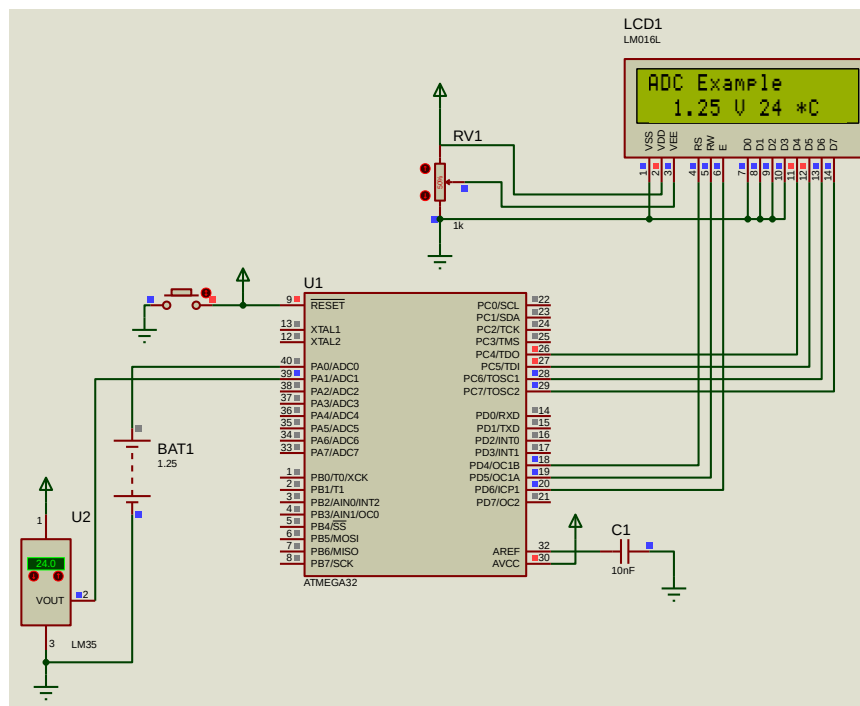


Figure 3: LM35 Temperature sensor Proteus Simulation

Discussion: The experiment shows how to interface an LM35 sensor with ATmega32 using its ADC. Averaging 32 samples improved reading accuracy. The measured voltage and temperature were displayed on an LCD. Proteus simulation verified the system's correct functionality.

Conclusion: LM35 interfacing with ATmega32 was successfully implemented and simulated. ADC readings were accurate and stable. LCD display and Proteus simulation worked as expected. The experiment met its objectives effectively.

Reference:

[1] Texas Instruments, "LM35 Precision Centigrade Temperature Sensor," Datasheet SNIS159G, 2000.